SITUAÇÃO DE APRENDIZAGEM

Modelagem e Desenvolvimento de Banco de Dados aplicada a um Sistema de Lanchonete

Componente Curricular: Banco de Dados — Carga Horária Total: 110h

1. Identificação da Atividade

Campo	Descrição
Componente curricular	Banco de Dados (40h)
Eixo de conteúdo	2. Modelo Entidade-Relacionamento (2.1 a 2.6 do Plano de Ensino) e módulo prático de desenvolvimento de sistemas (implementação)
Aulas de referência	----------
Capacidades técnicas mobilizadas	Aplicar os procedimentos do modelo de modelagem entidade-relacionamento; aplicar os procedimentos de normalização e padronização de dados; reconhecer as demandas do cliente; implementar um sistema web fullstack integrado a um banco de dados relacional; aplicar conceitos de conteinerização e implantação (deploy) de aplicações.
Capacidades socioemocionais mobilizadas	Valorizar novos fatos, ideias e opiniões diferentes na resolução de problemas; fundamentar escolhas e decisões a partir do exame de fatos, contextos e problemáticas de diferentes naturezas; trabalhar de forma colaborativa na integração entre modelagem de dados e desenvolvimento de software.
Organização da turma	Individual ou em duplas, a critério do professor
Instrumentos de avaliação previstos no plano	Exercícios de criação de Diagramas E-R (Formativa); Aplicação das Formas Normais (Formativa); Entrega de Modelo E-R para um estudo de caso (Somativa Parcial); Entrega do sistema web fullstack implantado via Docker Compose (Somativa Parcial).

2. Contexto

A Lanchonete Sabor & Cia iniciou suas atividades há três anos em um bairro de grande movimento e, desde então, cresceu de forma constante. O que começou como um pequeno balcão de atendimento local hoje já recebe pedidos presenciais, por telefone e também por um aplicativo próprio de delivery. Esse crescimento trouxe uma série de dificuldades: as comandas de pedidos ainda são anotadas em blocos de papel, o controle de estoque de ingredientes é feito “de cabeça” pelo responsável da cozinha, as promoções e combos mudam com frequência e ninguém sabe ao certo quais produtos venderam mais em cada período.

A proprietária da lanchonete contratou a sua equipe, uma consultoria de tecnologia, para projetar o banco de dados e desenvolver um sistema de gestão de pedidos, estoque e entregas. Antes de qualquer linha de código ser escrita, ela pediu que a equipe se reunisse com o pessoal do salão, da cozinha e da entrega para levantar como o negócio funciona hoje e como ele deve funcionar depois de informatizado.

Você foi designado(a) como responsável pela etapa de modelagem de dados e, na sequência, pela implementação de um sistema web funcional que dê suporte a essas regras. A proprietária não sabe o que é uma entidade, um atributo ou uma tabela — ela só sabe descrever como o seu negócio funciona. Cabe a você interpretar essa descrição, identificar as informações que precisam ser armazenadas, como elas se relacionam entre si, propor uma estrutura de banco de dados capaz de sustentar as regras descritas a seguir e, por fim, construir um sistema web que efetivamente utilize essa base de dados.

3. Comando

A partir do relato de funcionamento da lanchonete e das regras de negócio apresentadas na seção 4, você deverá:

- 1. Interpretar o cenário e identificar as entidades, os relacionamentos e as regras de cardinalidade e participação envolvidos no funcionamento da lanchonete;
- 2. Elaborar o Modelo Entidade-Relacionamento (MER) conceitual, representando entidades fortes, fracas e associativas, conforme aplicável;

- 3. Construir o Diagrama Entidade-Relacionamento (DER) correspondente, utilizando a notação apresentada em sala (Chen ou Pé-de-Galinha/Crow's Foot, conforme definido pelo professor), evidenciando cardinalidades, participação e tipos de atributos (simples, compostos, multivalorados, derivados, chaves);
- 4. Elaborar o Dicionário de Dados completo do modelo proposto;
- 5. Derivar o Modelo Lógico (esquema relacional) a partir do DER, aplicando as regras de conversão estudadas e verificando a normalização até, no mínimo, a 3ª Forma Normal (3FN);
- 6. Redigir uma justificativa técnica curta para pelo menos três decisões de modelagem tomadas (por exemplo: por que uma determinada informação virou entidade e não atributo, por que um relacionamento é N:N, por que uma entidade foi considerada fraca, etc.);
- 7. Implementar um sistema web fullstack a partir do Modelo Lógico proposto, com banco de dados relacional, backend e frontend (HTML, CSS e JavaScript), conforme detalhado no Módulo de Desenvolvimento (seção 7);
- 8. Realizar o deploy do sistema no servidor local disponível em sala, utilizando Docker Compose, com um container dedicado para o backend e outro para o banco de dados.

Este documento não indica quais entidades, atributos, tabelas ou tecnologias específicas devem ser criadas ou utilizadas. A leitura, interpretação e tradução das regras de negócio em um modelo de dados e em um sistema funcional são parte do que está sendo avaliado.

4. Regras de Negócio

As informações a seguir foram levantadas junto à proprietária, ao atendente do balcão, ao cozinheiro responsável pelo estoque e aos entregadores. Leia com atenção: cada regra carrega implicações para o modelo de dados e, posteriormente, para o sistema a ser desenvolvido.

4.1 Clientes e atendimento

- Um cliente pode fazer pedidos presencialmente no balcão, por telefone ou pelo aplicativo. O cadastro do cliente (nome, contato e, quando houver entrega, endereço) é obrigatório apenas para pedidos via aplicativo ou delivery; pedidos de balcão podem ser feitos sem identificar o cliente.
- Um mesmo cliente pode, ao longo do tempo, fazer vários pedidos, e a lanchonete quer conseguir consultar o histórico de pedidos de cada cliente cadastrado.
- Quando o pedido é feito para consumo no local, ele deve ser associado a uma mesa do salão; uma mesa pode receber vários pedidos ao longo do dia, mas cada pedido pertence a apenas uma mesa (ou a nenhuma, se for retirada ou delivery).

4.2 Cardápio, combos e variações

- O cardápio é dividido em categorias (lanches, bebidas, porções e sobremesas). Cada produto do cardápio possui um preço-base.
- Alguns produtos podem ser vendidos isoladamente ou fazer parte de um combo. Um combo reúne dois ou mais produtos do cardápio vendidos em conjunto por um preço promocional, diferente da soma dos preços individuais.
- Determinados produtos aceitam variações que alteram o preço final (por exemplo, tamanho do lanche ou tipo de ponto da carne) e adicionais opcionais escolhidos pelo cliente (por exemplo, bacon extra, queijo extra), que também têm preço próprio.
- A lanchonete quer poder identificar, para cada item vendido dentro de um pedido, quais adicionais e observações (“sem cebola”, “sem gelo”) foram escolhidos, pois isso impacta o preparo e o custo.

4.3 Pedidos e pagamento

- Um pedido é composto por um ou mais itens; cada item se refere a um produto (ou combo) do cardápio, em determinada quantidade.
- Um pedido pode ser pago com mais de uma forma de pagamento simultaneamente (por exemplo, parte em dinheiro e parte no cartão), sendo necessário registrar o valor pago em cada forma utilizada.
- Todo pedido possui um status que muda ao longo do tempo (recebido, em preparo, pronto, saiu para entrega, entregue/retirado, cancelado). A proprietária quer conseguir consultar, depois, em que horário cada mudança de status ocorreu, e não apenas o status atual.
- Pedidos podem utilizar um cupom de desconto, que tem um valor ou percentual de desconto, um período de validade e pode ser usado apenas uma vez por cliente.
- Ao final do atendimento, o cliente pode, opcionalmente, avaliar o pedido com uma nota e um comentário.

4.4 Entrega

- Pedidos feitos pelo aplicativo podem ser para retirada no balcão ou para entrega em um endereço informado pelo cliente.
- Quando há entrega, um entregador da equipe é responsável pelo trajeto. Um entregador pode ficar responsável por vários pedidos ao longo de um turno, mas cada pedido de delivery tem apenas um entregador responsável.
- A lanchonete cobra uma taxa de entrega que pode variar conforme a distância do endereço.

4.5 Equipe

- A equipe da lanchonete é composta por pessoas com diferentes cargos: atendente, cozinheiro, entregador e gerente. Uma mesma pessoa pode, eventualmente, acumular mais de uma função, mas a lanchonete quer manter o registro de qual cargo cada funcionário exerceu em cada período de trabalho.
- Cada pedido deve registrar qual atendente o recebeu, pois isso é usado para calcular comissões no fim do mês.

4.6 Estoque e ingredientes

- Cada produto do cardápio é preparado a partir de um ou mais ingredientes (insumos) do estoque, e um mesmo ingrediente pode ser usado em vários produtos diferentes. Para cada produto, é necessário saber a quantidade de cada ingrediente utilizada em seu preparo.
- Cada ingrediente tem uma quantidade em estoque, uma unidade de medida (quilo, litro, unidade) e um fornecedor responsável pelo abastecimento. Um fornecedor pode fornecer vários ingredientes diferentes.
- Ao registrar a venda de um item de pedido, a lanchonete quer futuramente ser capaz de estimar quanto de cada ingrediente foi consumido do estoque, com base na composição do produto vendido.

4.7 Promoções

- Além dos combos, a lanchonete cria promoções sazonais, que têm data de início e de término e podem se aplicar a um ou mais produtos do cardápio, alterando temporariamente o preço de venda.
- Um produto pode participar de mais de uma promoção ao longo do tempo, mas a lanchonete precisa saber quais promoções estiveram ativas em cada período, mesmo depois de encerradas.

5. Módulo de Desenvolvimento: Sistema Web Fullstack

Concluída a modelagem de dados, a proprietária pediu à consultoria que entregasse também uma primeira versão funcional do sistema, para validar com a equipe do salão, da cozinha e da entrega se o modelo proposto realmente sustenta a operação do dia a dia. Esta etapa complementa — e deve ser coerente com — o Modelo Lógico produzido na seção anterior: o sistema implementado deve utilizar exatamente (ou de forma muito próxima) o esquema relacional derivado do DER.

5.1 Visão geral da arquitetura

O sistema deve ser implementado como uma aplicação web fullstack, composta por três camadas claramente separadas:

- Frontend: interface web construída com HTML, CSS e JavaScript (puro ou com bibliotecas/frameworks leves, a critério do grupo e conforme orientação do professor), responsável por exibir os dados e permitir a interação do usuário (cadastrar, consultar, atualizar e excluir informações relevantes do negócio);
- Backend: API/servidor de aplicação responsável por implementar as regras de negócio, expor endpoints consumidos pelo frontend e se comunicar com o banco de dados. A linguagem/framework do backend é de livre escolha do grupo (por exemplo, Node.js/Express, PHP, Python/Flask ou Django, Java/Spring, entre outros), desde que aprovado pelo professor;
- Banco de Dados: SGBD relacional (por exemplo, MySQL, PostgreSQL ou MariaDB) que implementa fisicamente o Modelo Lógico entregue na seção anterior, incluindo chaves primárias, chaves estrangeiras e demais restrições definidas.

As três camadas devem se comunicar em containers separados, orquestrados via Docker Compose, conforme detalhado em 5.4.

5.2 Funcionalidades mínimas exigidas

O sistema não precisa cobrir 100% das regras de negócio da seção 4, mas deve demonstrar, de forma funcional, que o modelo de dados proposto sustenta a operação. No mínimo, o sistema deve permitir:

- Cadastro, consulta, edição e exclusão (CRUD) dos principais cadastros do negócio (por exemplo: clientes, produtos do cardápio, ingredientes, fornecedores e equipe);
- Registro de um pedido completo, associando cliente (quando aplicável), itens do pedido, produtos/combos, quantidade e forma(s) de pagamento;
- Atualização do status do pedido ao longo do tempo, preservando o histórico de mudanças de status (data/hora de cada transição);
- Alguma forma de consulta/relatório que utilize os relacionamentos do modelo (por exemplo: histórico de pedidos de um cliente, produtos mais vendidos em um período, ou consumo estimado de ingredientes).

O grupo pode, a critério do professor, negociar um subconjunto diferente de funcionalidades, desde que justificado tecnicamente e desde que o subconjunto escolhido evidencie o uso de pelo menos um relacionamento 1:N e um relacionamento N:N do modelo proposto.

5.3 Requisitos técnicos do frontend

- HTML semântico para a estrutura das páginas/telas;
- CSS próprio para a estilização (frameworks de CSS são permitidos como apoio, mas o grupo deve demonstrar domínio da estilização, não apenas o uso de um template pronto);
- JavaScript para interatividade e para o consumo da API do backend (requisições HTTP, por exemplo via fetch), incluindo tratamento básico de erros e feedback visual ao usuário (mensagens de sucesso/erro, carregamento, validação simples de formulários).

5.4 Deploy com Docker Compose no servidor local

A entrega final deve incluir a implantação (deploy) do sistema no servidor local disponível na sala de aula, utilizando Docker Compose, com a seguinte organização mínima de containers:

- Container do backend: responsável por executar a aplicação/API do servidor e servir (ou disponibilizar, conforme a arquitetura escolhida) o frontend;
- Container do banco de dados: responsável por executar o SGBD escolhido, com um volume nomeado para persistência dos dados entre reinicializações do container;
- Rede interna do Docker Compose conectando os containers, de modo que o backend acesse o banco de dados pelo nome do serviço (e não por localhost/IP fixo);
- Variáveis de ambiente (arquivo .env ou equivalente) para configuração de credenciais e parâmetros de conexão do banco de dados, evitando dados sensíveis “fixos” (hardcoded) no código;
- Mapeamento de portas do container do backend para uma porta do servidor local definida pelo professor/turma, de forma que o sistema fique acessível a partir de outras máquinas da rede da sala.

O grupo deve entregar um arquivo docker-compose.yml funcional na raiz do projeto, além de um Dockerfile para o backend (a imagem do banco de dados pode utilizar uma imagem oficial já existente, configurada via variáveis de ambiente e script de inicialização, quando necessário para criar o schema/dados iniciais).

Critério de aceite do deploy: ao executar “docker compose up” (ou “docker-compose up”) no servidor local da sala a partir da raiz do projeto entregue, o sistema deve subir sem erros e ficar acessível via navegador, com o frontend se comunicando corretamente com o backend e o backend se comunicando corretamente com o banco de dados em seu respectivo container.

5.5 Entregáveis do módulo de desenvolvimento

- Código-fonte completo do frontend, do backend e dos scripts de inicialização do banco de dados (incluindo o script DDL de criação das tabelas, coerente com o Modelo Lógico da seção 6.4);
- Dockerfile(s) e arquivo docker-compose.yml;
- Arquivo README.md com: instruções de como executar o projeto localmente (comando(s) de deploy), porta em que o sistema fica disponível, usuário/senha de teste (se houver autenticação) e uma breve descrição das funcionalidades implementadas;
- Prints de tela (ou vídeo curto) do sistema em funcionamento, demonstrando ao menos um cadastro, um pedido completo sendo registrado e uma consulta que evidencie um relacionamento do modelo.

6. Alinhamento com a Taxonomia de Bloom

Esta atividade foi estruturada para mobilizar progressivamente diferentes níveis cognitivos, conforme a Taxonomia de Bloom (revisada). O quadro a seguir relaciona cada nível às ações esperadas do estudante nesta situação de aprendizagem.

Nível	O que o estudante deve fazer nesta atividade
Compreender	Interpretar o relato de funcionamento da lanchonete e as regras de negócio, reconhecendo fatos, processos e necessidades de informação implícitas no texto.
Aplicar	Aplicar os conceitos de entidade, atributo, relacionamento e cardinalidade estudados em sala para traduzir o cenário em um MER e em um DER, e aplicar conceitos de desenvolvimento web e conteinerização para implementar e implantar o sistema.
Analisar	Analisar as regras de negócio para decidir o que é entidade, o que é atributo, quais relacionamentos existem, suas cardinalidades e participações, e quais dependências funcionais existem entre os dados, aplicando a normalização; analisar como o modelo lógico se traduz em um sistema funcional.
Avaliar	Justificar tecnicamente as decisões de modelagem e de implementação tomadas, avaliando alternativas possíveis (por exemplo, transformar um relacionamento em entidade associativa, ou escolher determinada tecnologia de backend) e argumentando por que a escolha feita atende melhor às regras de negócio.
Criar	Elaborar, de forma original, o conjunto completo de artefatos do projeto: MER, DER, Dicionário de Dados, Modelo Lógico normalizado e um sistema web fullstack funcional, implantado via Docker Compose, coerentes entre si.

7. O que deve ser entregue

A entrega final deve conter, obrigatoriamente, os cinco blocos de artefatos abaixo, coerentes entre si (ou seja, o DER deve refletir o MER descrito, o dicionário de dados deve descrever exatamente os elementos do DER, o modelo lógico deve ser derivado do mesmo DER, e o sistema implementado deve utilizar esse mesmo modelo lógico).

7.1 Modelo Conceitual (MER)

- Descrição textual e/ou esquemática das entidades identificadas, classificando-as como fortes, fracas ou associativas, quando for o caso;
- Identificação dos relacionamentos entre as entidades, com suas cardinalidades (1:1, 1:N, N:N) e tipo de participação (total ou parcial), com justificativa baseada nas regras de negócio;
- Identificação dos atributos relevantes de cada entidade, indicando quais são chave, compostos, multivalorados ou derivados, quando existirem.

7.2 Diagrama Entidade-Relacionamento (DER)

- Representação gráfica do MER utilizando a notação definida em sala de aula;
- Deve conter todas as entidades, atributos, relacionamentos, cardinalidades e chaves primárias/estrangeiras necessárias para atender às regras de negócio descritas;
- Recomenda-se o uso de uma ferramenta de modelagem (por exemplo, brModelo, MySQL Workbench, dbdiagram.io, draw.io/diagrams.net ou similar).

7.3 Dicionário de Dados

- Para cada entidade do modelo, uma tabela contendo, no mínimo: nome do atributo, tipo de dado, tamanho/precisão quando aplicável, se é chave (primária, estrangeira, candidata), se é obrigatório, e uma breve descrição do significado do dado;
- O dicionário deve ser suficiente para que outra pessoa, sem conhecer o projeto, entenda o significado de cada dado armazenado.

7.4 Modelo Lógico

- Conversão do DER em esquema relacional, com a definição das tabelas, colunas, tipos de dados, chaves primárias e estrangeiras e demais restrições (obrigatoriedade, unicidade, valores permitidos);
- Verificação e, se necessário, ajuste do esquema para eliminar dependências parciais e transitivas, garantindo no mínimo a 3ª Forma Normal (3FN);
- Pode ser apresentado como diagrama relacional, como script de criação (DDL) ou como conjunto de tabelas descritas em notação relacional (Nome_Tabela(coluna1, coluna2, ...)), conforme orientação do professor.

7.5 Sistema Web Fullstack (Módulo de Desenvolvimento)

- Código-fonte do frontend (HTML, CSS, JavaScript), do backend e dos scripts de banco de dados;
- Dockerfile(s) e docker-compose.yml funcionais, com deploy validado no servidor local da sala;
- README.md com instruções de execução e credenciais de teste;
- Evidências (prints ou vídeo curto) do sistema em funcionamento, conforme especificado em 5.5.

8. Uso de Inteligência Artificial nesta Atividade

O uso de ferramentas de IA como apoio ao estudo é permitido e incentivado para tirar dúvidas conceituais, revisar sua lógica e discutir alternativas de modelagem ou de implementação. No entanto, o código-fonte, o MER, o DER, o dicionário de dados e o modelo lógico entregues devem ser produção do(a) próprio(a) estudante (ou da dupla). Trabalhos identificados como gerados integralmente por IA, sem domínio das decisões tomadas pelo(a) estudante, poderão ser desconsiderados para fins de avaliação, a critério do professor.

9. Requisitos de Entrega

Item	Descrição
Formato dos arquivos	Um único arquivo compactado (.zip) contendo: DER em imagem ou PDF; dicionário de dados em planilha, tabela de documento ou PDF; modelo lógico em diagrama, script SQL ou documento; MER e justificativas em documento de texto (PDF ou .docx); e uma pasta “sistema/” com o código-fonte completo do frontend, backend, Dockerfile(s), docker-compose.yml e README.md do módulo de desenvolvimento.
Organização da entrega	Individual ou em dupla, conforme definição do professor em sala.
Ferramentas sugeridas (modelagem)	brModelo, MySQL Workbench, dbdiagram.io, draw.io/diagrams.net, Lucidchart ou outra ferramenta de modelagem aprovada pelo professor.
Ferramentas/tecnologias sugeridas (desenvolvimento)	Frontend: HTML5, CSS3, JavaScript. Backend: Node.js/Express, PHP, Python (Flask/Django) ou outra stack aprovada pelo professor. Banco de dados: MySQL, PostgreSQL ou MariaDB. Conteinerização: Docker e Docker Compose.
Deploy	Obrigatório no servidor local disponível na sala de aula, via “docker compose up”, com container dedicado para o backend e container dedicado para o banco de dados, acessível pela rede local da sala.
Prazo	A ser definido pelo professor no cronograma da turma.
Forma de entrega	Upload no ambiente pedagógico indicado pelo professor (Situação de Aprendizagem associada), acompanhado da demonstração do sistema em funcionamento no servidor local em data combinada com o professor.
Natureza da avaliação	Somativa parcial (Entrega do Modelo E-R para o estudo de caso e entrega/demonstração do sistema implantado), precedida pelas avaliações formativas de exercícios de DER e aplicação das Formas Normais previstas no plano de ensino.